

MC714 – Sistemas Distribuídos

Laboratório 2 – Implementação de Algoritmos Distribuídos

Wallysson Oliveira Pasqualini – RA 200061

Luana Amorim Lima – RA 197620

6 de julho de 2026

Repositório: <https://github.com/Sholum1/mc714-laborat-rio-2/tree/main>

Sumário

1	Introdução	2
2	Descrição do Problema	2
3	Tecnologias Utilizadas	2
4	Arquitetura da Solução	3
5	Relógio Lógico de Lamport	3
6	Exclusão Mútua Distribuída	4
7	Eleição de Líder	5
8	Recurso Compartilhado	6
9	Interface de Comandos	6
10	Ambiente de Execução	6
11	Experimentos Realizados	6
12	Resultados	7
13	Discussão	7
14	Conclusão	7
15	Referências	8
16	Fontes Externas e Adaptações	8

1 Introdução

Este relatório descreve a implementação de algoritmos distribuídos desenvolvida para o segundo trabalho da disciplina Sistemas Distribuídos (MC714). O objetivo da atividade é implementar, utilizando alguma forma de comunicação distribuída, três mecanismos clássicos: relógio lógico de Lamport, um algoritmo de exclusão mútua e um algoritmo de eleição de líder ou consenso distribuído.

A solução implementada utiliza Guile Scheme e a biblioteca Goblins para representar processos distribuídos como atores. A comunicação entre nós é realizada por meio de referências remotas fornecidas pelo mecanismo OCapN/CapTP, utilizando uma camada de rede TCP/TLS. Cada nó executa como um processo independente, podendo ser conectado aos demais por meio de sturdyrefs.

Os algoritmos implementados foram:

- Relógio lógico de Lamport;
- Algoritmo de Ricart–Agrawala para exclusão mútua distribuída;
- Algoritmo Bully para eleição de líder.

2 Descrição do Problema

Em sistemas distribuídos, os processos executam de forma independente e se comunicam apenas por troca de mensagens. Como não há memória compartilhada nem relógio global perfeitamente sincronizado, surgem problemas de coordenação, ordenação de eventos e controle de acesso a recursos compartilhados.

Neste trabalho, o sistema foi projetado para simular uma rede de nós distribuídos capazes de trocar mensagens, manter relógios lógicos, disputar acesso a uma seção crítica e eleger um coordenador. A implementação busca demonstrar, de forma prática, como algoritmos estudados em sala podem ser implementados sobre uma infraestrutura real de comunicação entre processos.

3 Tecnologias Utilizadas

A implementação utiliza as seguintes tecnologias:

- **Linguagem:** Guile Scheme;
- **Modelo de concorrência:** atores, por meio da biblioteca Goblins;
- **Comunicação distribuída:** OCapN/CapTP sobre TCP/TLS;
- **Ambiente de execução:** GNU Guix;
- **Isolamento de rede:** namespaces de rede Linux, interfaces virtuais *veth* e *bridge br0*;
- **Sistema operacional:** GNU/Linux.

Além do código principal dos algoritmos, foram utilizados scripts auxiliares para configurar e limpar o ambiente de rede. O script de configuração cria uma bridge, associa interfaces virtuais aos namespaces dos processos e atribui endereços IP aos nós. O script de limpeza remove os namespaces e a bridge criados durante a execução.

4 Arquitetura da Solução

A solução é composta por múltiplos nós independentes. Cada nó possui um conjunto de atores responsáveis por funcionalidades específicas: relógio lógico, registro de peers, exclusão mútua, eleição de líder e acesso ao recurso compartilhado.

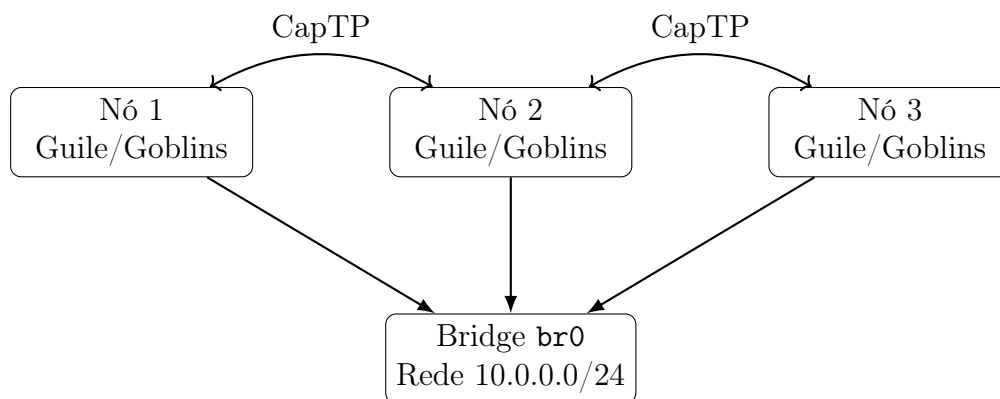


Figura 1: Arquitetura geral da solução distribuída.

Cada processo inicia seu próprio nó e registra uma referência remota. Essa referência, chamada de *sturdyref*, é impressa no terminal e pode ser usada pelos demais nós para estabelecer conexões. Após conectados, os nós podem trocar mensagens diretamente, iniciar eleições, solicitar entrada na seção crítica e acessar o recurso compartilhado.

5 Relógio Lógico de Lamport

O relógio lógico de Lamport foi implementado como um ator responsável por manter um contador inteiro local. Esse contador representa o tempo lógico do processo. A cada evento local relevante, como o envio de uma mensagem, o relógio é incrementado. Ao receber uma mensagem de outro nó, o processo atualiza seu relógio para o máximo entre o valor local e o timestamp recebido, somando uma unidade em seguida.

A regra implementada segue a definição clássica:

$$C_i = \max(C_i, C_j) + 1$$

onde C_i é o relógio local do processo receptor e C_j é o timestamp recebido na mensagem.

Na implementação, o ator do relógio possui três operações principais: **tick**, usada para incrementar o relógio antes de eventos locais ou envios; **receive**, usada para atualizar o relógio ao receber uma mensagem; e **peek**, usada para consultar o valor atual.

```
(define-actor (~lamport-clock bcom time)
  (methods
```

```

((tick)
  (let ((new-time (+ time 1)))
    (bcom (~lamport-clock bcom new-time) new-time)))
((receive remote-time)
  (let ((new-time (+ (max time remote-time) 1)))
    (bcom (~lamport-clock bcom new-time) new-time)))
((peek) time)))

```

Esse relógio é utilizado tanto nas mensagens comuns de chat quanto nas mensagens do algoritmo de exclusão mútua. Dessa forma, cada requisição de acesso à seção crítica carrega um timestamp lógico, permitindo ordenar requisições concorrentes.

Referências utilizadas. A implementação deste componente foi baseada na descrição formal do algoritmo proposta por Leslie Lamport e em materiais de apoio sobre sistemas distribuídos. A implementação foi desenvolvida em Guile Scheme utilizando o modelo de atores da biblioteca Goblins e adaptada à arquitetura deste projeto.

- Lamport, L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 1978.
- https://en.wikipedia.org/wiki/Lamport_timestamp

6 Exclusão Mútua Distribuída

Para o problema de exclusão mútua, foi implementado o algoritmo de Ricart–Agrawala. O objetivo desse algoritmo é garantir que apenas um processo por vez tenha acesso à seção crítica, sem a necessidade de um coordenador central.

Cada nó pode estar em um de três estados principais:

- **idle:** o nó não está interessado na seção crítica;
- **wanting:** o nó solicitou acesso e aguarda respostas dos demais;
- **held:** o nó possui o acesso à seção crítica.

Quando um nó deseja acessar a seção crítica, ele incrementa seu relógio lógico, envia uma mensagem `mutex-request` para todos os peers e passa para o estado `wanting`. Cada outro nó decide se responde imediatamente ou se adia a resposta. A decisão é baseada no timestamp da requisição e no identificador do processo. Requisições com menor timestamp têm prioridade. Em caso de empate, o menor identificador tem prioridade.

Quando o processo recebe respostas suficientes dos demais peers, ele obtém acesso ao recurso compartilhado. Na implementação, esse acesso é representado por um token. O token permite escrever no recurso compartilhado, e é revogado quando o nó libera a seção crítica.

```

((lock! callback)
  (match state
    ('idle
      (let ((ts ($ clock 'tick)))
        ($ peers 'broadcast! (list 'mutex-request ts my-id))
        (bcom (~mutex-participant bcom my-id clock peers resource

```

```
        '(wanting ,ts 0 ,callback ())))))
    (_ (format #t "Already requesting/holding lock.~%"))))
```

Ao liberar a seção crítica, o nó envia as respostas que haviam sido adiadas durante o período em que estava aguardando ou mantendo o acesso. Isso garante que as próximas requisições possam prosseguir.

Referências utilizadas. A implementação do algoritmo de Ricart–Agrawala foi baseada na descrição formal apresentada pelos autores e em materiais de referência sobre exclusão mútua distribuída. O algoritmo foi adaptado ao modelo de atores da biblioteca Goblins e à comunicação via CapTP utilizada neste projeto.

- Ricart, G.; Agrawala, A. *An Optimal Algorithm for Mutual Exclusion in Computer Networks*. Communications of the ACM, 1981.
- https://en.wikipedia.org/wiki/Ricart%E2%80%93Agrawala_algorithm

7 Eleição de Líder

O algoritmo de eleição escolhido foi o Bully. Nesse algoritmo, cada processo possui um identificador, e o processo com maior identificador entre os ativos deve ser escolhido como líder.

Quando um nó inicia uma eleição, ele consulta a lista de peers conhecidos e envia mensagens de eleição apenas para os processos com identificadores maiores que o seu. Se nenhum processo maior existir, o próprio nó se declara líder e envia uma mensagem `coordinator` para os demais. Se algum processo de identificador maior responde com `election-ok`, o nó aguarda uma mensagem de coordenador. Caso essa mensagem não chegue dentro de um timeout, a eleição é reiniciada.

```
((elect! . maybe-cb)
 (let* ((cb (if (pair? maybe-cb) (car maybe-cb) pending-cb))
        (new-round (+ round 1))
        (all-ids ($ peers 'list-ids))
        (higher (filter (lambda (i) (string>? i my-id)) all-ids)))
  (if (null? higher)
      (begin
        (format #t "I am the leader (id=~a).~%" my-id)
        ($ peers 'broadcast! (list 'coordinator my-id)))
      ...)))
```

A implementação também mantém informações sobre a fase atual da eleição, o líder conhecido, a rodada da eleição e o tempo restante até o timeout. Essas informações podem ser consultadas pelo comando `election`.

Referências utilizadas. A implementação do algoritmo Bully foi baseada na descrição proposta por Garcia-Molina e em materiais de referência sobre eleição de líder em sistemas distribuídos. A solução foi adaptada para integração com a arquitetura baseada em atores da biblioteca Goblins e com a comunicação via CapTP.

- Garcia-Molina, H. *Elections in a Distributed Computing System*. IEEE Transactions on Computers, 1982.

- https://en.wikipedia.org/wiki/Bully_algorithm

8 Recurso Compartilhado

O recurso compartilhado foi implementado como um ator que armazena um valor. O acesso de escrita ao recurso só pode ser feito por meio de um token de acesso. Esse token é criado quando o algoritmo de exclusão mútua concede acesso à seção crítica.

A leitura do recurso é realizada por meio de um leitor criado pelo próprio recurso. Já a escrita exige que o nó tenha obtido o lock previamente. Assim, a implementação separa explicitamente operações de leitura e escrita, e protege a escrita por meio da exclusão mútua distribuída.

9 Interface de Comandos

A aplicação possui uma interface textual interativa. Os principais comandos disponíveis são:

Comando	Descrição
<code>connect <sturdyref></code>	Conecta o nó a outro peer
<code>send <id> <texto></code>	Envia mensagem para outro nó
<code>lock</code>	Solicita acesso à seção crítica
<code>unlock</code>	Libera a seção crítica
<code>host-resource</code>	Cria e hospeda um recurso compartilhado
<code>connect-resource <sref></code>	Conecta-se a um recurso compartilhado
<code>write <texto></code>	Escreve no recurso, caso possua o token
<code>read</code>	Lê o valor atual do recurso
<code>elect</code>	Inicia uma eleição de líder
<code>leader</code>	Mostra o líder conhecido
<code>status</code>	Mostra o estado local do nó

Tabela 1: Comandos disponíveis na aplicação.

10 Ambiente de Execução

Para executar múltiplos nós de forma isolada, foram utilizados namespaces de rede do Linux. Um script auxiliar cria uma bridge chamada `br0`, configura o endereço `10.0.0.1/24` e conecta cada processo a essa bridge por meio de pares de interfaces virtuais `veth`. Cada nó recebe um endereço IP na rede `10.0.0.0/24`.

Esse ambiente permite testar a comunicação distribuída em uma única máquina, mas preservando o isolamento de rede entre os nós. Assim, é possível observar o comportamento da solução como se cada nó estivesse executando em uma máquina independente.

11 Experimentos Realizados

Foram realizados testes com múltiplos nós conectados entre si. Os principais experimentos foram:

1. Inicialização de múltiplos nós e conexão por sturdyrefs;
2. Envio de mensagens entre nós, verificando a atualização dos relógios de Lamport;
3. Criação de um recurso compartilhado em um dos nós;
4. Conexão dos demais nós ao recurso compartilhado;
5. Solicitação de lock por diferentes nós;
6. Escrita no recurso somente após obtenção do token de acesso;
7. Liberação da seção crítica e atendimento de requisições pendentes;
8. Inicialização de eleição de líder e verificação do líder escolhido.

12 Resultados

Nos testes de troca de mensagens, foi possível observar que os relógios lógicos eram incrementados no envio e atualizados corretamente no recebimento. Isso permitiu preservar a relação causal entre eventos comunicantes.

Nos testes de exclusão mútua, apenas o nó que havia recebido todas as permissões necessárias conseguia obter o token de acesso e escrever no recurso compartilhado. Quando outro nó solicitava o lock simultaneamente, a prioridade era decidida pelo timestamp lógico e, em caso de empate, pelo identificador do processo.

Nos testes de eleição, o algoritmo Bully escolheu como líder o nó de maior identificador entre os conhecidos. Quando um nó de identificador menor iniciava a eleição, ele enviava mensagens para os nós de identificador maior e aguardava a resposta. Caso não houvesse nós maiores, ele se declarava coordenador.

13 Discussão

A implementação permitiu observar na prática algumas dificuldades típicas de sistemas distribuídos. Uma delas é a necessidade de lidar com comunicação assíncrona, pois mensagens podem ser respondidas posteriormente por meio de callbacks e vows. Outra dificuldade é manter o estado de cada algoritmo de forma consistente, especialmente no caso da exclusão mútua, em que o processo pode estar ocioso, aguardando permissões ou mantendo a seção crítica.

O uso de atores ajudou a organizar a solução, pois cada componente do nó foi modelado como uma entidade independente. Por outro lado, isso também exigiu cuidado no encadeamento de operações assíncronas, principalmente nos casos de acesso remoto, concessão de token e timeouts de eleição.

14 Conclusão

O trabalho implementou três mecanismos fundamentais de sistemas distribuídos: relógios lógicos, exclusão mútua e eleição de líder. A solução utiliza Guile Scheme, Goblins e comunicação remota por CapTP sobre TCP/TLS, permitindo que múltiplos nós se comuniquem em um ambiente distribuído.

Os experimentos demonstraram o funcionamento dos algoritmos implementados e reforçaram conceitos importantes da disciplina, como ordenação causal, coordenação sem memória compartilhada, controle de acesso a recursos compartilhados e escolha de coordenador em um conjunto de processos.

Como melhorias futuras, seria possível adicionar testes automatizados, registrar logs estruturados dos eventos, simular falhas de nós de forma mais controlada e melhorar a visualização das mensagens trocadas durante a execução dos algoritmos.

15 Referências

1. Lamport, L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 1978.
2. Ricart, G.; Agrawala, A. *An Optimal Algorithm for Mutual Exclusion in Computer Networks*. Communications of the ACM, 1981.
3. Garcia-Molina, H. *Elections in a Distributed Computing System*. IEEE Transactions on Computers, 1982.
4. Goblins Documentation. <https://spritely.institute/goblins/>
5. OCapN / CapTP Documentation. <https://files.spritely.institute/docs/ocapn/>
6. Goblins Source Code. <https://gitlab.com/spritely/goblins>

16 Fontes Externas e Adaptações

Durante o desenvolvimento deste trabalho foram consultados os artigos originais dos algoritmos implementados e materiais de apoio sobre sistemas distribuídos, incluindo a documentação oficial da biblioteca Goblins e páginas de referência que descrevem o funcionamento dos algoritmos. A implementação foi desenvolvida pelos autores em Guile Scheme e integrada ao modelo de atores fornecido pela biblioteca Goblins.

Foram realizadas adaptações para possibilitar:

- integração entre os algoritmos de relógio lógico, exclusão mútua e eleição de líder;
- comunicação distribuída utilizando CapTP sobre TCP/TLS;
- gerenciamento de recursos compartilhados por meio de atores;
- interface de linha de comando para interação com os nós;
- execução em um ambiente isolado utilizando namespaces de rede Linux.

Dessa forma, o projeto consiste em uma implementação própria baseada nas descrições formais dos algoritmos e adaptada à arquitetura específica desenvolvida para este trabalho.